MODULE 03 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda I – The Execution Model

How an execution environment is born, reused and destroyed - and why almost every Lambda question turns on that.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 3 of 34

admin@sthithapragna.com

Lambda does not run your function. It builds an **execution environment**, and then runs your function inside it, possibly many times.

The three phases of an execution environment

01

INIT

Download code, start the runtime, run your initialisation code. Only on a cold start.

02

INVOKE

Run the handler. Repeats for every request the warm environment serves.

03

SHUTDOWN

Environment is reclaimed. Lambda decides when, not you.

ON THE EXAM

INIT happens once per environment. INVOKE happens many times. Knowing which phase code runs in is the most useful Lambda skill you can have.

What actually happens during INIT

- **Extension init.** Any external extensions start first and register themselves.
- **Runtime init.** The language runtime bootstraps. Node, Python, Java and so on.
- **Function init.** Your module-level code runs. Imports, clients, config, connections.
- The whole INIT phase gets up to **10 seconds** before Lambda restarts it.
- A cold start is INIT plus the first INVOKE. Only INIT is avoidable.

> ON THE EXAM
>
> Heavy imports and eager client construction are function init work. That is where cold start latency is won or lost.

The most important line in your code

OUTSIDE THE HANDLER

Runs once per environment

- Imports and module-level code
- SDK clients and database connections
- Config and secrets fetched at startup
- Expensive setup you want amortised
- Billed as INIT duration

INSIDE THE HANDLER

Runs on every single request

- Per-request business logic
- Anything that depends on the event
- Anything that must not be shared
- Billed as INVOKE duration, every time
- Runs thousands of times, not once

THE TIEBREAKER

If it is the same for every request, put it outside. If it depends on the event, it must go inside. Exam questions about latency or cost very often hinge on code sitting on the wrong side of this line.

What survives between invocations

- **Global and static variables.** Values set during INIT are still there.
- **Open connections.** Database and HTTP connections stay alive and reusable.
- **Files written to /tmp.** They persist for the life of the environment.
- **Background processes are frozen**, not killed, and resume on the next invoke.
- None of this is guaranteed. Lambda may hand you a fresh environment at any time.

> ON THE EXAM
>
> Reuse is an optimisation you may exploit but never depend on. Correct code works identically warm or cold.

The traps this creates

- **Leaked state.** A global mutated by request A is visible to request B.
- **Cached credentials or tenant context** bleeding across users. A real security bug.
- **/tmp filling up** over a long-lived environment until writes start failing.
- **Concurrency multiplies everything.** 200 concurrent requests means 200 environments, each with its own globals, its own /tmp and its own connections.
- A connection pool of 10 in code becomes 2,000 connections at 200 concurrency.

> ON THE EXAM
>
> The fourth point is the one that catches people. Environment reuse and concurrency pull in opposite directions.

Ephemeral storage: the /tmp directory

PROPERTY	VALUE
Path	/tmp - the only writable location in the filesystem
Default size	512 MB
Configurable	512 MB to 10,240 MB, in 1 MB increments
Cost	Free up to 512 MB, charged per GB-second above that
Lifetime	Lives and dies with the execution environment
Shared?	No. Each concurrent environment has its own /tmp

ON THE EXAM

Scratch space for one invocation, sized up to 10 GB. Not shared storage, not durable storage, not a cache you can rely on.

/tmp or Amazon EFS?

CHOOSE /TMP WHEN

Scratch space for one invocation

- Temporary files nobody else needs
- Up to 10 GB is enough
- You want zero extra configuration
- No VPC required
- Fastest option - it is local disk

CHOOSE EFS WHEN

Shared or persistent state

- Several functions need the same files
- You need more than 10 GB
- Data must outlive the environment
- Requires VPC configuration and a mount target
- Adds cold start and networking overhead

THE TIEBREAKER

Temporary and private means /tmp. Shared or persistent means EFS. If the scenario says the files are not needed afterwards, /tmp is almost always the answer.

Memory is the only performance dial

- **128 MB to 10,240 MB**, in 1 MB increments.
- **CPU scales in proportion to memory.** There is no separate CPU setting.
- At **1,769 MB** a function gets the equivalent of one full vCPU.
- Network bandwidth scales with memory too.
- Under 1,769 MB you have a fraction of a vCPU, so multi-threading buys you nothing.

> ON THE EXAM
>
> More memory can make a function cheaper. If doubling memory more than halves duration, the bill goes down.

arm64 and Graviton

- Lambda runs on **x86_64** or **arm64**, the latter backed by AWS Graviton.
- arm64 is typically **cheaper per GB-second** and often faster for the same work.
- Switching is a configuration change for interpreted runtimes like Python and Node.js.
- Native binaries and compiled dependencies must be rebuilt for the target architecture.
- Layers and container images are architecture-specific. They must match the function.

> ON THE EXAM
>
> When a question asks for cheaper compute with no code rewrite, arm64 is a strong candidate - unless native dependencies are mentioned.

What you actually pay for

- **Requests.** A flat charge per invocation, regardless of duration.
- **Duration in GB-seconds.** Memory configured, multiplied by time billed.
- **Ephemeral storage** above the free 512 MB, also per GB-second.
- Billed in **1 ms increments**, so shaving milliseconds genuinely pays at scale.
- **CloudWatch Logs is billed separately** and is a real cost on chatty functions.

> ON THE EXAM
>
> GB-seconds is the number that matters. Memory x duration, which is why the two dials trade against each other.

The hard limits that decide questions

LIMIT	VALUE
Function timeout	900 seconds, that is 15 minutes. Default 3 seconds
Memory	128 MB to 10,240 MB
Ephemeral storage	512 MB to 10,240 MB
Environment variables	4 KB total, all variables combined
Layers per function	5
Deployment package	50 MB zipped upload, 250 MB unzipped including layers
Container image	10 GB
Payload, synchronous	6 MB request and 6 MB response
Payload, asynchronous	256 KB

ON THE EXAM

These are hard limits. No support ticket raises them. Concurrency is different - that is a soft limit and can be increased.

Timeout behaviour

- **Default is 3 seconds.** Easy to forget, and a common cause of mystery failures.
- **Maximum is 900 seconds.** Fifteen minutes. Not negotiable.
- On timeout the invocation is killed. No graceful shutdown, no cleanup handler.
- Set the timeout slightly above expected duration, not at the maximum.
- A timeout downstream of API Gateway still costs you the full billed duration.

> ON THE EXAM
>
> Work longer than 15 minutes means Step Functions, ECS, Fargate or AWS Batch. Module 13 covers choosing between them.

Environment variables

- **4 KB total** across every variable on the function, combined.
- Encrypted at rest by default with an AWS managed KMS key.
- Use a **customer managed key** when you need to control or audit access.
- Values are **visible in the console** to anyone with GetFunctionConfiguration.
- So they are fine for config, and wrong for secrets.

> ON THE EXAM
>
> Endpoint URLs and table names belong here. Database passwords belong in Secrets Manager. Module 19 covers the split properly.

Lambda layers

- **Up to 5 layers** per function, extracted to **/opt** at startup.
- Layers count towards the **250 MB unzipped** package limit. They do not bypass it.
- Versioned and immutable. A function pins a specific layer version ARN.
- Shareable across accounts, or made public, via layer permissions.
- Layers do not reduce cold start time. The content still has to be downloaded.

> ON THE EXAM
>
> Layers are for sharing and for keeping deployment packages small. They are not a workaround for the size limit.

ZIP archive or container image?

CHOOSE ZIP WHEN

The default

- Package fits inside 250 MB unzipped
- You want the simplest deployment
- You want to use layers
- Generally faster cold starts
- Editable in the console for small functions

CHOOSE CONTAINER IMAGE WHEN

Big, or already containerised

- Up to 10 GB uncompressed
- Large ML models or heavy dependencies
- Your team already builds images
- Stored in Amazon ECR, not S3
- Layers are not supported

THE TIEBREAKER

The 250 MB ceiling is the usual decider. Above it, container images are the only option - and you give up layers to get there.

Lambda extensions

- **Internal extensions** run inside the runtime process, altering how it starts.
- **External extensions** run as separate processes alongside your function.
- They start before INIT and can keep running after the handler returns.
- Typical uses: monitoring agents, secret caching, configuration fetching.
- They share the function's memory, CPU and cost. They are not free.

> ON THE EXAM
>
> Extensions are the supported way to run an agent inside Lambda without embedding it in your handler code.

The Parameters and Secrets extension

- AWS-provided layer that caches **Secrets Manager** and **Parameter Store** values.
- Your function calls a **localhost HTTP endpoint** instead of the AWS API.
- Cuts latency and cuts API calls, which cuts cost and throttling risk.
- Cache TTL is configurable, so rotated secrets are picked up.
- Without it, fetching a secret inside the handler means an API call on every request.

> ON THE EXAM
>
> When a scenario complains about latency or cost from repeated Secrets Manager calls, this extension is very often the intended answer.

Execution role or resource-based policy?

EXECUTION ROLE

What the function may do

- An IAM role the function assumes
- Grants access to DynamoDB, S3, KMS and so on
- Always required, even for logging
- Credentials appear in the environment automatically
- Outbound permissions

RESOURCE-BASED POLICY

Who may invoke the function

- A policy attached to the function itself
- Lets S3, EventBridge or API Gateway call it
- Enables cross-account invocation
- Limited to 20 KB
- Inbound permissions

THE TIEBREAKER

Outbound is the execution role. Inbound is the resource-based policy. An 'access denied' when a service tries to trigger your function is almost always a missing resource-based policy.

How a function logs

- Everything written to stdout or stderr lands in **/aws/lambda/<function-name>**.
- **The execution role must allow it.** No permission means no logs, and no error either.
- Advanced logging controls set the **log format** to JSON and a **log level**, without code changes.
- JSON format lets Logs Insights query fields directly instead of parsing text.
- You can send logs to a **custom log group**, useful for separating environments.

> ON THE EXAM
>
> A function that appears to produce no logs at all is usually a missing CreateLogStream or PutLogEvents permission, not a broken function.

Lambda inside a VPC

- By default a function runs **outside your VPC** with internet access.
- Attach it to a VPC to reach private resources like RDS or ElastiCache.
- **A VPC-attached function loses internet access** unless you add a NAT gateway.
- For AWS services, **VPC endpoints** are cheaper and faster than a NAT gateway.
- Hyperplane ENIs are shared across environments, so VPC attachment no longer adds much cold start cost.

> ON THE EXAM
>
> The classic failure: a developer attaches a function to a VPC to reach RDS, and its calls to S3 or a public API start timing out.

Reading the REPORT line

FIELD	WHAT IT TELLS YOU
Duration	How long the handler actually ran
Billed Duration	What you pay for. Now includes Init Duration
Memory Size	What you configured
Max Memory Used	What was actually consumed - the right-sizing signal
Init Duration	Cold start work. Present only on a cold start

ON THE EXAM

Max Memory Used far below Memory Size means you are over-provisioned. Init Duration appearing often means cold starts are frequent.

Currency note: INIT is now billed

- Before, functions packaged as **ZIP with a managed runtime** did not pay for INIT on on-demand invocations.
- From **1 August 2025**, INIT duration is billed across every configuration.
- Provisioned Concurrency, custom runtimes and container images always billed it.
- AWS reports INIT occurs in under 1% of invocations for most workloads.
- Mitigations: smaller packages, SnapStart, or Provisioned Concurrency.

> ON THE EXAM
>
> Older cost material and older practice questions predate this. Cold starts used to be partly free. They are not any more.

Runtimes and deprecation

- Managed runtimes follow the upstream language's support lifecycle.
- After deprecation you **cannot create** functions on that runtime.
- Later, you cannot **update** existing ones either. They keep running, unpatched.
- AWS emails owners and posts dates well ahead. Notices land in Health Dashboard.
- A **custom runtime** via the Runtime API is the escape hatch for unsupported languages.

> ON THE EXAM
>
> The examinable point is the order: create is blocked first, update second, execution never. Existing functions are not switched off.

Powertools for AWS Lambda

- An AWS-maintained library for Python, TypeScript, Java and .NET.
- **Structured logging** in JSON, with the request ID attached automatically.
- **Custom metrics** via embedded metric format, with no extra API call.
- **Tracing** helpers, and an **idempotency** utility backed by DynamoDB.
- Distributed as a Lambda layer, so adding it is a configuration change.

> ON THE EXAM
>
> When a question wants structured logs, custom metrics or idempotency with the least development effort, Powertools is the low-effort answer.

Answer each one **before** the walkthrough plays. Reading a solution you did not commit to teaches almost nothing.

A developer builds a Lambda function that reads from an Amazon RDS for PostgreSQL database. Following best practice, they create the database connection outside the handler so it is reused across invocations. In testing with light traffic everything works and latency is good.

In production the function is invoked heavily and concurrency reaches around 900. The database begins rejecting connections with 'too many clients already', and the function starts failing. The database instance has a max_connections setting of 500.

What explains the failure, and what is the best fix?

A The connection outside the handler is a bug. Move it inside so each invocation closes its own connection

B Each concurrent execution environment holds its own connection, so ~900 environments means ~900 connections. Put Amazon RDS Proxy in front of the database

C Increase the function's memory so connections are established faster

D Increase the RDS instance size so max_connections rises above 900

◆ PAUSE AND DECIDE

What explains the failure, and what is the best fix?

A Move the connection inside the handler

This makes it worse, not better. You still get one connection per concurrent execution, but now you pay to establish it on every single request.

B One connection per environment; add RDS Proxy

Correct diagnosis and the purpose-built fix. RDS Proxy pools and multiplexes connections so thousands of Lambda environments share a small pool.

C Increase memory

Memory buys CPU and bandwidth. It has no effect on how many connections exist.

D Scale the database up

It would work today and fail again at higher concurrency. It treats the symptom and pays for a bigger instance to do it.

THE SIGNAL

Reuse is per environment, not per function. Concurrency multiplies every per-environment resource.

A media company runs a Lambda function that downloads a source video from Amazon S3, transcodes it, and uploads the result back to S3. Source files are up to 6 GB, and the transcoding library requires the whole file on a local filesystem. It cannot stream.

The work completes in about 9 minutes. The intermediate files are useless once the upload finishes. The team wants the simplest configuration that works, and does not want to manage a VPC.

How should the developer provide the working storage?

A. Configure the function's ephemeral storage to 10,240 MB and work in /tmp

B. Attach an Amazon EFS file system to the function and write there

C. Attach an Amazon EBS volume to the function

D. Stream the file from S3 in chunks and keep only a small buffer in /tmp

◆ PAUSE AND DECIDE

How should the developer provide the working storage?

A Ephemeral storage at 10,240 MB

6 GB fits inside the 10 GB maximum, the files are temporary and private, and it needs no VPC. Simplest thing that works.

B Amazon EFS

It would work, but it requires VPC configuration and a mount target, which the scenario explicitly rules out. Reach for EFS when files are shared or must persist.

C Amazon EBS

EBS cannot be attached to a Lambda function at all. It is block storage for EC2.

D Stream in chunks

Directly contradicts a stated constraint - the library requires the whole file on disk and cannot stream.

THE SIGNAL

Two sentences decided this: files are not needed afterwards, and no VPC. Both point at /tmp.

A data-processing Lambda function is configured with 512 MB of memory and takes 12 seconds per invocation. It is CPU-bound: it parses and compresses a large payload in memory. It runs 5 million times a month and the bill has become the team's largest serverless line item.

A developer tests the same function at 2,048 MB and finds it completes in 2.5 seconds. Max Memory Used reports 340 MB in both configurations.

What should the team conclude?

A Keep 512 MB. Max Memory Used is only 340 MB, so 2,048 MB wastes memory they do not need

B Move to 2,048 MB for the speed, and accept that the bill will rise roughly fourfold

C Move to 2,048 MB. It is faster and cheaper, because 4x the memory for less than a quarter of the duration lowers total GB-seconds

D Enable Provisioned Concurrency, since the slowness is caused by cold starts

◆ PAUSE AND DECIDE

What should the team conclude?

A Keep 512 MB, memory is barely used

The trap. Memory allocation buys CPU, not just RAM. Max Memory Used tells you nothing about how much CPU the function needs.

B Move up and pay 4x

Right action, wrong arithmetic. Cost is memory multiplied by duration, and the duration fell by more than the memory rose.

C Move up: faster and cheaper

512 MB x 12s = 6 GB-seconds. 2,048 MB x 2.5s = 5 GB-seconds. Lower cost and a 5x speed-up.

D Provisioned Concurrency

Solves cold starts. Nothing here points to cold starts - the duration is consistent, and the fix worked by adding CPU.

THE SIGNAL

Cost is memory multiplied by duration. For CPU-bound work, more memory routinely costs less.

> An API-backed Lambda function retrieves a database credential from AWS Secrets Manager. The call sits at the top of the handler. The function is invoked about 400 times per second at peak.
>
> The team observes two problems: roughly 60 ms of added latency on every request, and intermittent ThrottlingException errors from Secrets Manager. The credential is rotated every 30 days. Security requires that a rotated credential is picked up without redeploying.

Which change best addresses both problems?

A. Move the secret into a Lambda environment variable encrypted with a customer managed KMS key

B. Move the Secrets Manager call outside the handler so it runs once per execution environment

C. Cache the secret in a module-level variable with no expiry, populated on first use

D. Add the AWS Parameters and Secrets Lambda Extension and read the secret from its local endpoint, with a cache TTL shorter than the rotation period

◆ PAUSE AND DECIDE

Which change best addresses both problems?

A Encrypted environment variable

Environment variables are readable by anyone with GetFunctionConfiguration, and the value would freeze at deploy time, breaking the rotation requirement.

B Move the call outside the handler

A real improvement and the closest wrong answer. But a long-lived environment caches the value indefinitely, so a rotated secret may never be picked up.

C Module-level cache, no expiry

Same defect as B, stated explicitly. No expiry means no rotation. It also still calls Secrets Manager once per cold start.

D Parameters and Secrets extension with a TTL

Removes the per-request API call, so latency and throttling both go away, and the TTL guarantees rotated values are collected without a deployment.

THE SIGNAL

Three constraints, not two: latency, throttling, and rotation without redeploy. Only the TTL satisfies the third.

What to carry into Lambda II

- Environment reuse is **per environment**. Concurrency multiplies every per-environment resource.
- The handler boundary decides whether code runs **once** or **every time**.
- Memory buys CPU. More memory is often **cheaper**, not dearer.
- Hard limits: 15 minutes, 10,240 MB memory, 10 GB /tmp, 4 KB env vars, 5 layers.
- Outbound permission is the execution role. Inbound is the resource-based policy.

> ON THE EXAM
>
> Next: concurrency, scaling and cold starts - where reserved and provisioned concurrency, SnapStart and throttling all live.